

LAB EXPERIMENT REPORT

Data Structures using Java

Basic Information	
Experiment No.	_____
Date	_____
Title	Balanced Parentheses Checker using Stack in Java

1. Objective / Aim

To write a Java program that checks whether the brackets `(){}[]` in a given string are correctly matched and properly nested, by using the Stack data structure and its LIFO (last-in, first-out) behaviour.

2. System Requirements

Hardware Requirements

- Processor: Intel Core i3 / AMD equivalent or higher
- RAM: 4 GB minimum (8 GB recommended)
- Hard disk: 500 MB free space
- Input/Output: Keyboard, monitor

Software Requirements

- Operating System: Windows 10/11, Linux or macOS
- Java Development Kit (JDK) 8 or higher
- Editor / IDE: Eclipse, IntelliJ IDEA, NetBeans or VS Code
- Command Prompt / Terminal for compilation and execution

3. Algorithm / Logic Description

Key idea: Every closing bracket must match the *most recently opened* bracket that is still unclosed. A stack stores opening brackets in order; the top of the stack is always the latest unmatched one.

Algorithm:

1. Start.
2. Read the input string and create an empty stack of characters.
3. Take the next character `ch` from the string.
4. If `ch` is an opening bracket `{ [ (`, push it onto the stack.
5. If `ch` is a closing bracket `) ] }`: if the stack is empty, the string is invalid (stop); otherwise pop the top and check that it is the matching opening bracket. If not, the string is invalid (stop).
6. Ignore any other character.
7. Repeat steps 3-6 until all characters are processed.

8. If the stack is empty, the string is valid; otherwise some opening brackets were never closed, so it is invalid.

9. Display the result. Stop.

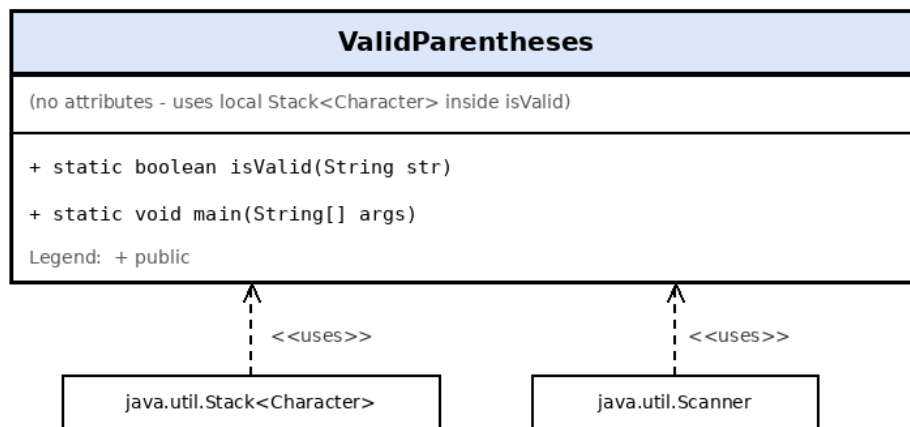
Trace for {[()]}:

Char	Action	Stack (bottom to top)
{	push	{
[	push	{ [
(	push	{ [(
)	pop (- matches	{ [
]	pop [- matches	{
}	pop { - matches	(empty) - Valid

Complexity: Time $O(n)$, Space $O(n)$ in the worst case, where n is the length of the string.

Note: Characters other than brackets are skipped, so an input such as `a(b)c` is reported as valid. An empty string is also reported as valid.

4. UML Class Diagram



5. Source Code

```
import java.util.*;

public class ValidParentheses {

    public static boolean isValid(String str) {

        Stack<Character> stack = new Stack<>();

        for (char ch : str.toCharArray()) {

            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            }

            else if (ch == ')' || ch == '}' || ch == ']') {

                if (stack.isEmpty()) {
                    return false;
                }

                char top = stack.pop();

                if ((ch == ')' && top != '(') ||
                    (ch == '}' && top != '{') ||
                    (ch == ']' && top != '[')) {

                    return false;
                }
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter parentheses: ");
        String str = sc.nextLine();

        if (isValid(str)) {
            System.out.println("Valid Parentheses");
        } else {
            System.out.println("Invalid Parentheses");
        }

        sc.close();
    }
}
```

6. Compilation and Execution Commands

Save the program as **ValidParentheses.java** (the file name must match the public class). Open a terminal in that folder and run:

```
javac ValidParentheses.java
java ValidParentheses
```

7. Output

```
Terminal - Valid Parentheses

$ javac ValidParentheses.java
$ java ValidParentheses
Enter parentheses: {[()]}
Valid Parentheses

$ java ValidParentheses
Enter parentheses: ([)]
Invalid Parentheses

$ java ValidParentheses
Enter parentheses: ((
Invalid Parentheses

$ java ValidParentheses
Enter parentheses: ()[]{}
Valid Parentheses

$ _
```

Input	Expected	Actual Output
{[()]}	Valid Parentheses	Valid Parentheses
(D]	Invalid Parentheses	Invalid Parentheses
((	Invalid Parentheses	Invalid Parentheses
()[]{}	Valid Parentheses	Valid Parentheses

8. Result / Conclusion

The program correctly identifies balanced and unbalanced bracket strings using a stack. It detects all three error types: a closing bracket with no opener, mismatched bracket pairs, and unclosed opening brackets left on the stack.